Accelerate Native Sparse Attention Kernels on Blackwell using CUTLASS Python

Akash Mehra, Sr. Gen AI Algorithms Engineer
Vincent Zhang, Sr. Software Engineer

GTC 2026 | March 18

Agenda

Basic Introduction of Cutlass Python

NSA Overview

Compression Attention

TopK Kernel

Selection Attention

End-to-End Performance Comparison

CUTLASS in Python

Brief overview

A Python-based domain-specific language designed for dynamic compilation of high-performance GPU kernels.

Core Programming Model:

- **`@cute.jit` Decorator**
  - JIT-compiled functions
  - Callable from Python or other DSL functions
  - Automatic IR caching for repeated calls
  - Entry point for kernel orchestration
- **`@cute.kernel` Decorator**
  - GPU kernel functions
  - Compiled as specialized GPU symbols
  - Launched via @jit functions
  - Direct tensor core access

C++ Alignment: Semantics match CuTe C++

Framework Integration: DLPack support for PyTorch, JAX interoperability

Type Inference: Automatic type deduction reduces boilerplate

Performance: Faster compilation and very close in performance to CUTLASS C++

Why CUTLASS in Python

Fine-Grained Hardware Control

- **Direct Tensor Core Programming:** tcgen05.mma, WGMMA
- **Explicit Pipelining:** Multi-stage producer/consumer
- **Async Data Movement:** TMA, shared memory orchestration
- **Custom Layouts:** Explicit tile layouts and swizzling for peak memory bandwidth

Developer Productivity

- **Python Environment:** Familiar syntax, tooling
- **Faster Compilation:** JIT vs C++ templates
- **Instant Experimentation:** Edit code, see results in seconds
- **Maintains Performance:** Performance comparable to CUTLASS C++

NSA Overview

Native Sparse Attention (NSA)

Approach

Attention Module is divided into three key branches.

Input to the module are queries, keys and values

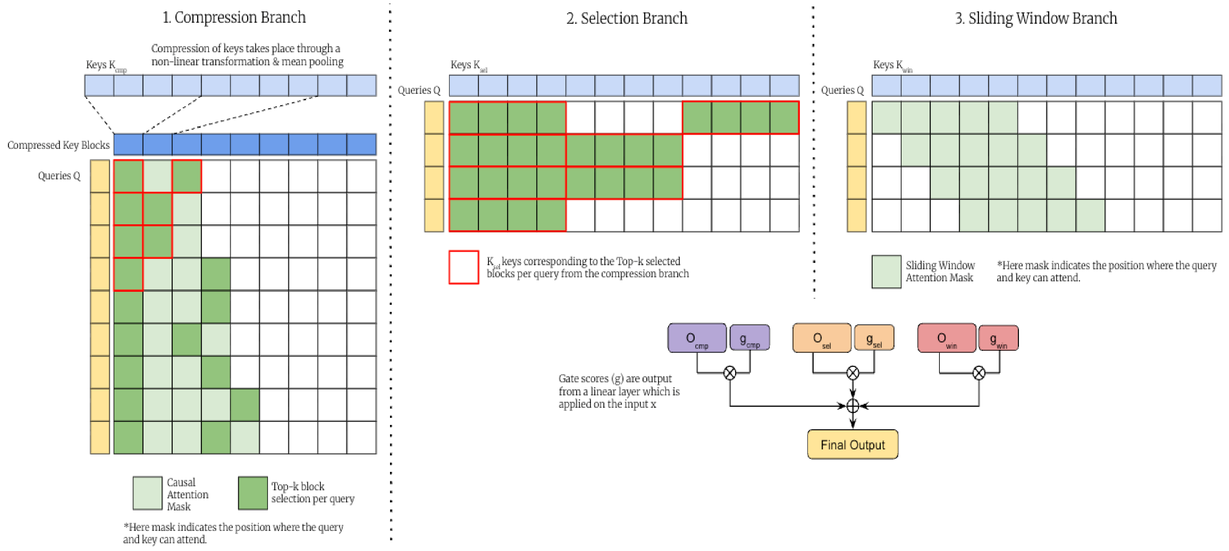
Compressed Attention computes Attention from **compressed** keys and values.

TopK indices are selected based on importance scores derived from compressed attention scores.

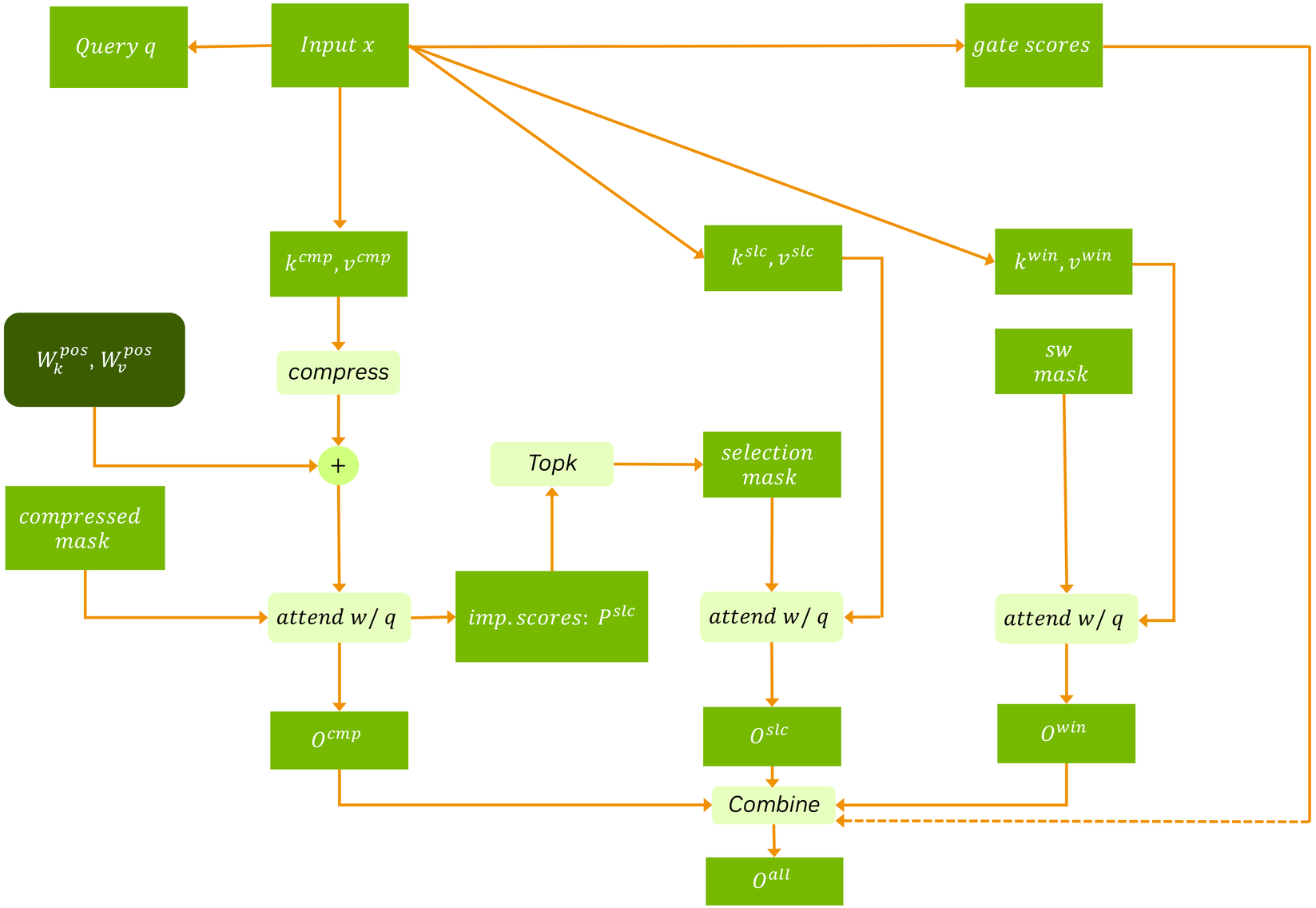
These selected indices load keys and values for a sparse Attention computation called Selection Attention.

In another branch, Sliding window attention captures local relationships.

Outputs from the three branches are then combined and sent to the next module.

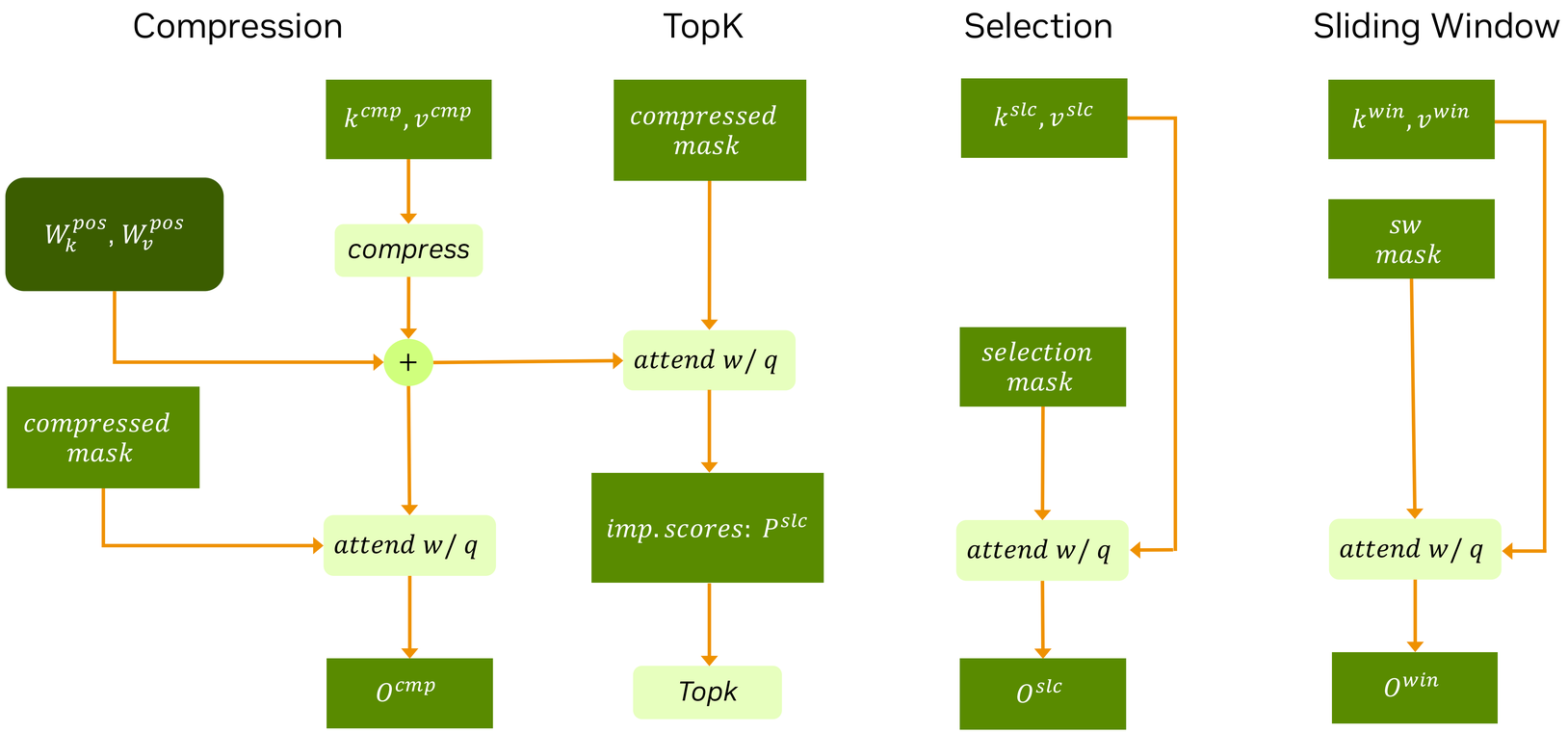


NSA Architecture Overview



Required Kernels

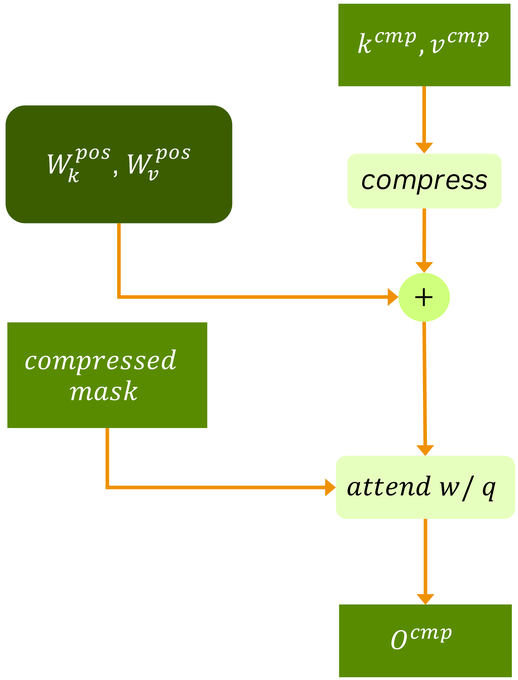
Forward & Backward



Compression Attention

Compression Attention Overview

Compression



Keys and Values are compressed after projection

Hyper-parameters:

- Compression block size
- Compression block stride
- In our implementation they are the same, so no overlap.

We add position information before calculating a compressed statistic about a block of tokens

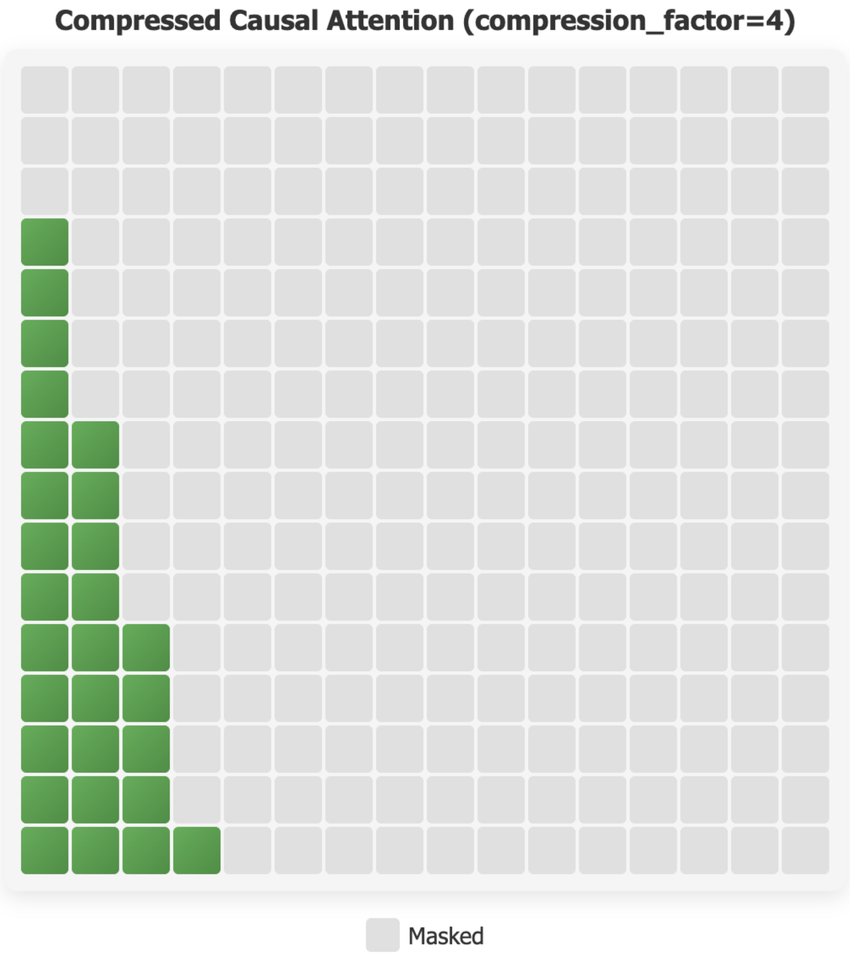
Perform GQA version of Fused Muti-head Attention

Compression mask is more restrictive than causal mask

- Each query token is projected into the compressed block and can only attend to the block if its projected coords are strictly greater than the block idx

Compressed Causal Mask

Example and Visualization



Different than a causal mask when key sequence length < query sequence length

Staircase pattern means we'll be applying more mask per trip count compared uncompressed case

Few pipeline trips e.g., seqlen_q = 8192, seqlen_k = 256 results in only 2 trips with tiler_mn = (128, 128)

Lower arithmetic intensity because of the non-squared GEMM.

Applying the mask is not for free and costs time.

Since the pipeline trip counts are so low, each pipeline stage is mostly exposed with little pipeline stage overlapping.

Even for perfect overlap, the instructions used in mask application(FSEL, ISETP, VIADD) compete for some of the same resources (instruction issue) as the MUFU.EX2 instructions.

Optimization Journey: Improving Masked Compression

- **Problem:** Initial masked compression implementation was significantly slower than unmasked baseline
- **Setup:** batch=4, q_heads=64, k_heads=16, head_dim=128

s_q / s_k	Flop Count	Unmasked, Persistent (µs)		TFlop/s	Masked, Persistent (µs)		Masked, Non-Persistent (µs)		Masked, Non-Persistent, Improved Masking (µs)		Masked, Non-Persistent, Improved Masking, mma_tiler 128,64 (µs)	
4096 / 128	6.87E+10	167	100%	411	222	133%	247	148%	220	132%	209	125%
8192 / 256	2.75E+11	298	100%	922	820	275%	662	222%	527	177%	476	160%
16384 / 512	1.10E+12	919	100%	1196	2168	236%	1803	196%	1365	149%	1199	130%
64000/ 1000	4.19E+12	3828	100%	1096	6552	174%	5490	143%	3655	95%	3269	85%
132000/ 2000	1.68E+13	13508	100%	1242	22675	168%	19278	143%	11408	84%	10394	77%

- Persistent scheduling was was slower compared to non-persistent as there was a load imbalance
- Reducing control flow overhead improved performance
- Smaller K-tiles even though reduced MMA efficiency, but it also meant we had to apply less masking

TopK Kernel

TopK Kernel

Design choice

Question:

- Should we save Compression scores from Compression Attention kernels
  - Use CopyReduceBulkTensorTileS2GOp to reduce all heads
  - Save them to HBM
  - Load them from HBM
  - Compute importance scores
  - Call torch.topk or online TopK

Observations:

- FMHA is explicitly a fused kernel to avoid materializing the attention scores to GMEM
- Materializing Compression scores requires writing them to HBM from softmax warps
- Re-introduced a GMEM round-trip on a hot path.

Solution:

- Recomputing Q * K.T avoids making the kernel more memory bound
- We also just need indices; we can just use scores instead of probabilities.
- But for making backward relatively simpler, we need topk_scores to be a probability distribution.

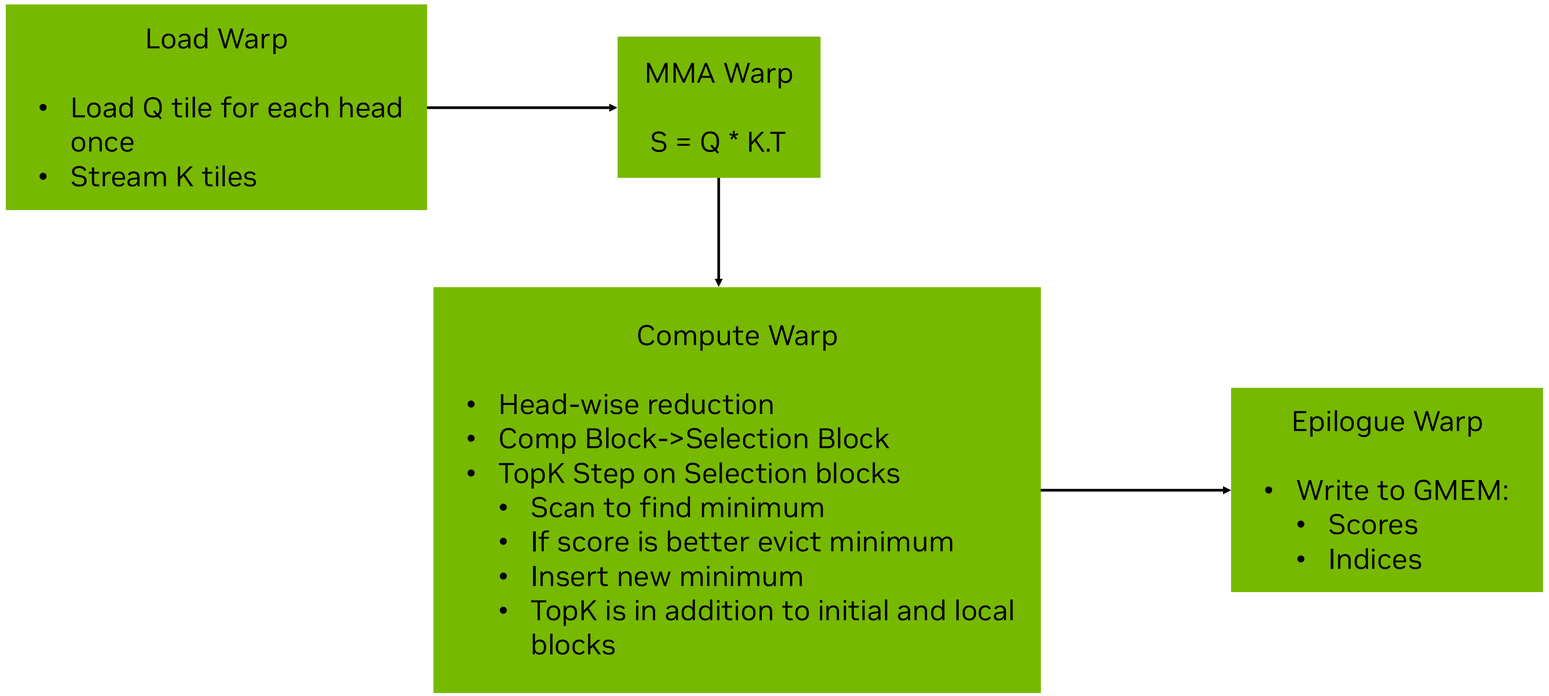
TopK Kernel

Forward

Design:

- Keep one Q tile resident in the CTA and iterate over all K tiles.
- This allows us to see all keys and select the top K blocks online.

CTA Structure (launched per query group):



TopK Kernel

Backward

```
# d: how many compressed K positions one selection block covers
d = selection_block_size // compress_stride
row_sums  = (topk_scores * dSsel).sum(dim=k, keepdim=True)                # (s_q, 1, h_k, b)
dSsel_grad = topk_scores * (dSsel - row_sums) * scale                     # (s_q, k, h_k, b)
indices_d = topk_indices[..., None] * d + arange(d)                       # (s_q, k, h_k, b, d)
indices_d = indices_d.reshape(s_q, k*d, h_k, b)
dScmp = zeros(s_q, s_k, h_k, b)
src = rearrange(dSsel_grad[..., None].expand(s_q, k, h_k, b, d), 'q k h b d -> q (k d) h b’)
dScmp.scatter_(dim=-1, index=indices_d, src=src)
dScmp = dScmp.repeat_interleave(h_r, dim=h_k)                             # (s_q, s_k, h_q, b)
Q     = rearrange(Q, 'q h d b -> q d h b’)
K_rep = rearrange(K.repeat_interleave(h_r, dim=1), 'k h d b -> k d h b’)
dK_full = einsum('q k h b, q d h b -> k d h b', dScmp, Q_r)
dK      = rearrange(dK_full, 'k d (hk hr) b -> k d hk hr b’).sum(dim=3)   # (s_k, d, h_k, b)
dQ      = einsum('q k h b, k d h b -> q d h b', dScmp, K_rep)             # (s_q, d, h_q, b)
```

Selection Attention

Selection Attention

Algorithm

- Problem Size [h_k, seq_len, b]

```
for Q_tile_i in Q_tiles: # in parallel
    load Q_tile_i: [gqa_size, d]
    O = zeros_like(Q_tile_i)
    for j in topn:
        get kv_idx from block_indices[j]
        load K_tile_j, V_tile_j according kv_idx
        S = Q_tile_i @ K_tile_j
        max_old = row_max
        row_max = max(S, dim=-1)
        P = softmax(S)
      O = P @ V_tile_j + O * exp(max_old - max_new)
    Store O to smem
    TMA store O
```

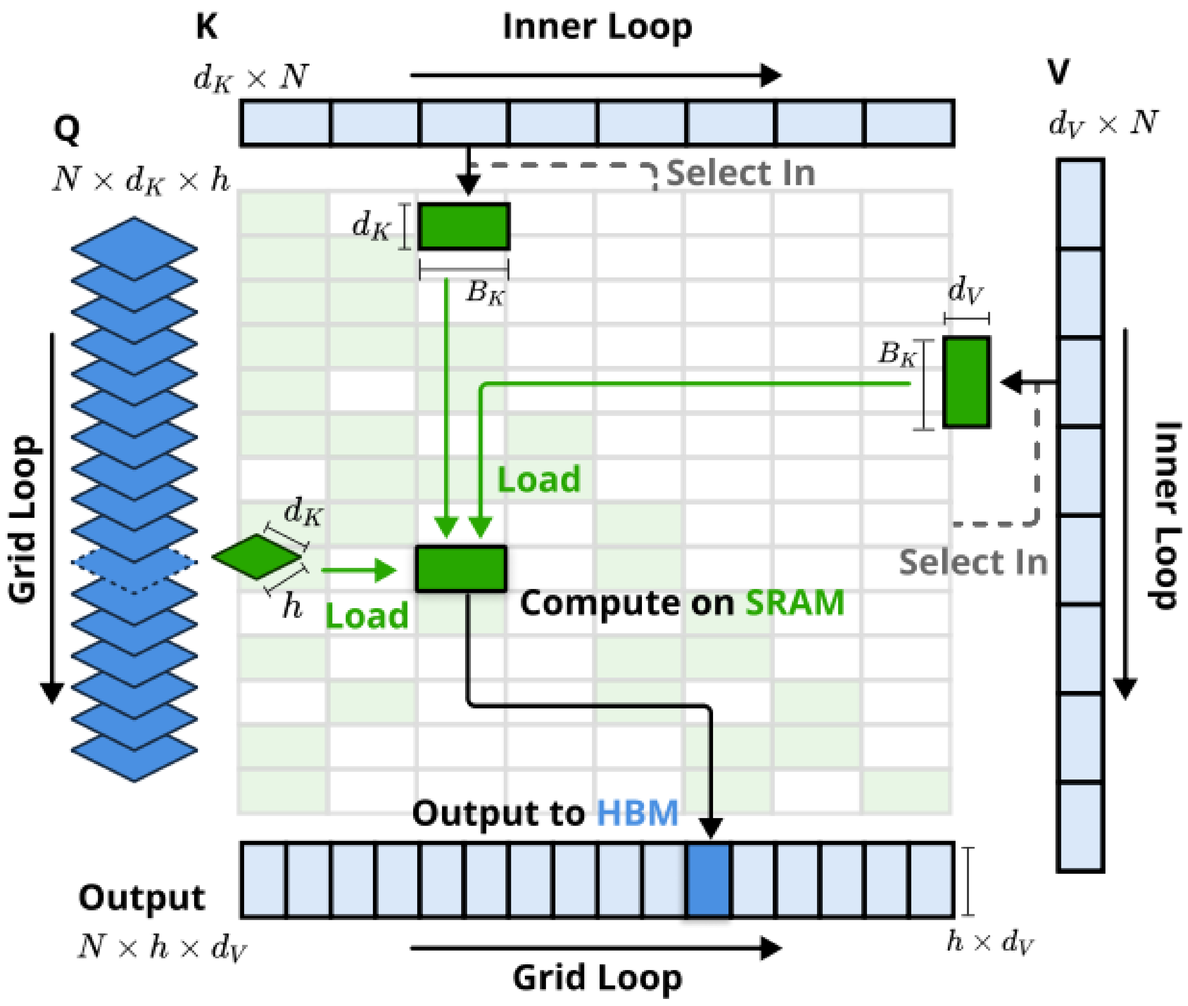


Image from “Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention”

Design Challenges Compared to Standard MHA

Standard MHA

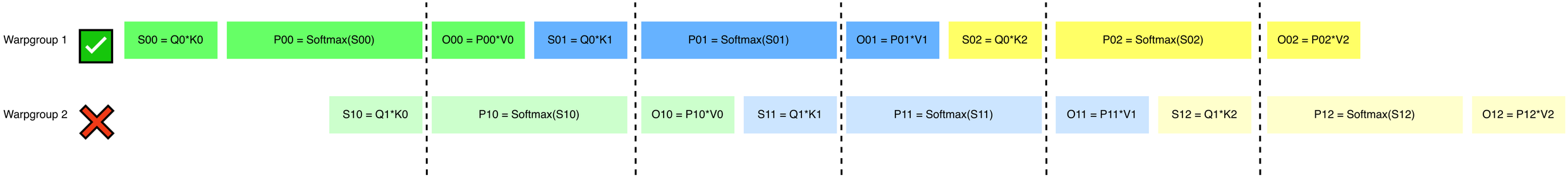
- Different query tile attend to the same set of K/V block with causal mask
- Once K/V tile is loaded into shared memory, it can be shared across different query tile in each CTA
- Q/K/V tile can be as large as hardware permits(e.g. 128x128), which can maximize the tensor core efficiency

NSA Selection Attention

- Different query group(within the same GQA group) attends to different K/V block
- K/V tile cannot be shared across different query group
- Q tile is limited to GQA size(e.g. 16), and K/V tile is limited to selection block size(e.g. 64)

Design limitations in NSA selection attention

- Cannot use two warp-group strategy
- Limited tile size degrade tensor core efficiency

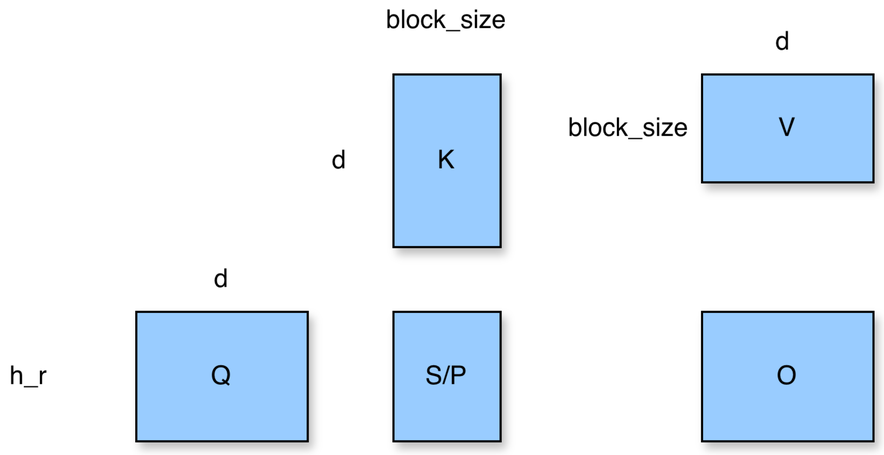


As Q0 and Q1 tile doesn't attend to the same K/V tile set, two warp-group pingpong strategy cannot be applied

Pack Two K/V Block

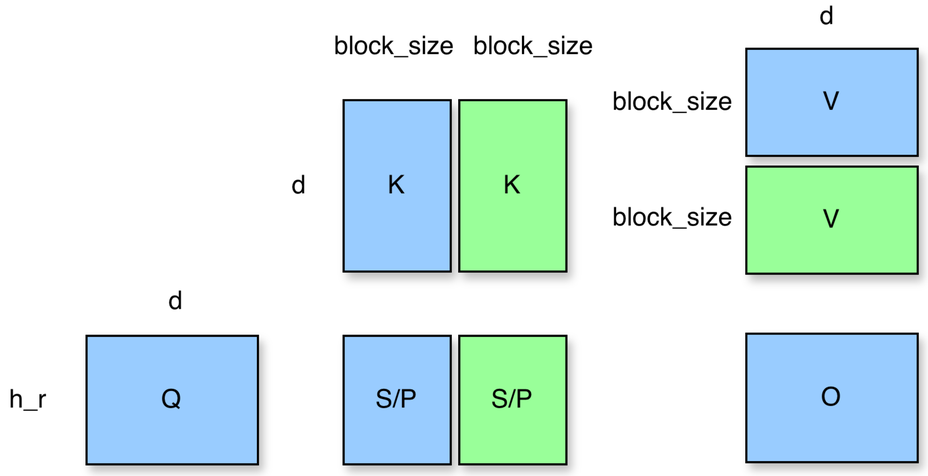
Why do we need to pack K/V tile

- For m=128, n=64, k=128
  - Load A(smem): 256 cycles
  - Load B(smem): 128 cycles
  - MMA: 256 cycles
  - Load A/B = 384 cycles, cannot be overlapped by MMA Inst



Pack two K/V blocks into one

- For m=128, n=128, k=128
  - Load A(smem): 256 cycles
  - Load B(smem): 256 cycles
  - MMA: 512 cycles
  - Load A/B = 512 cycles, can be overlapped by MMA Inst
  - **Bonus**: A larger tile size can reduce the number of corrections and alleviate the load on the CUDA cores.



Pack Two K/V Block

When the selected block number = an odd number

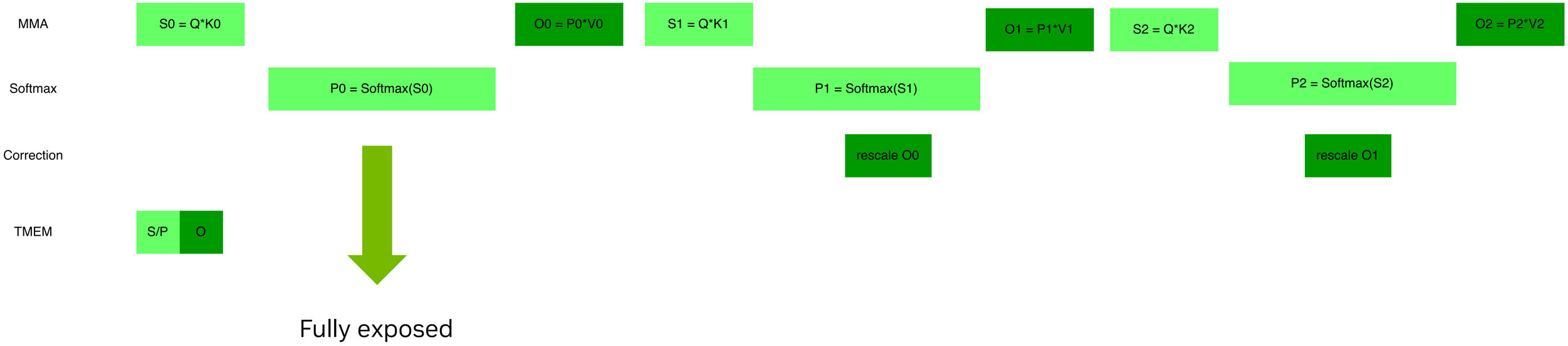
- E.g. selected block number = 5, the packed k/v block = [0, 1], [2, 3], [4, x]

- Tricks to handle invalid block in packed blocks
  - Using cute.domain_offset to offset gK/gV to out of bounds, the TMA will fill zeros
    - K and V will be zero
    - P * V will be zero
  - Mask the second block when
    - do reduce_max to get the max value
    - do reduce_sum to get the row sum

Intra-warp Overlap

Non-overlap pipeline

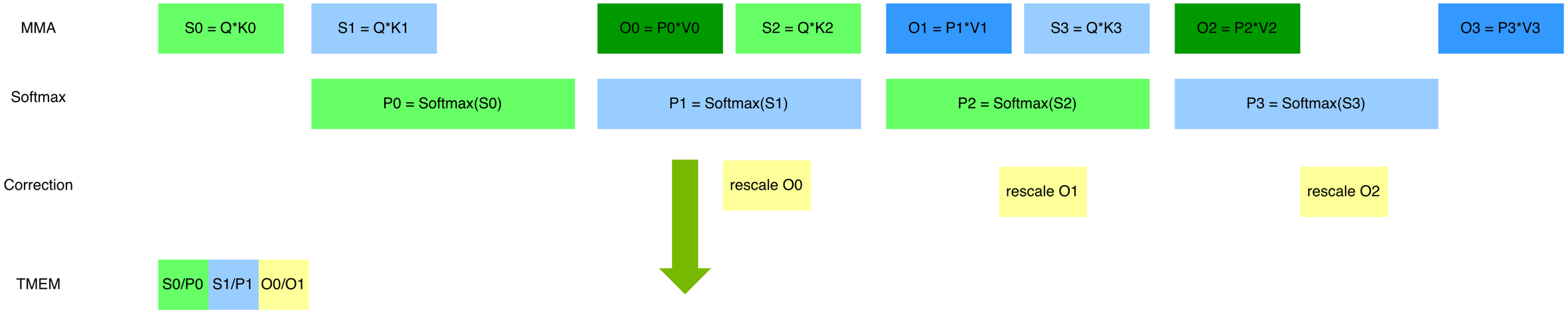
- Single softmax stage
- Serialized pipeline: QK -> Softmax –> PV -> QK -> …
- Softmax operation is fully exposed
- Maximum 50% compute throughput



Intra-warp Overlap

Intra-warp overlap

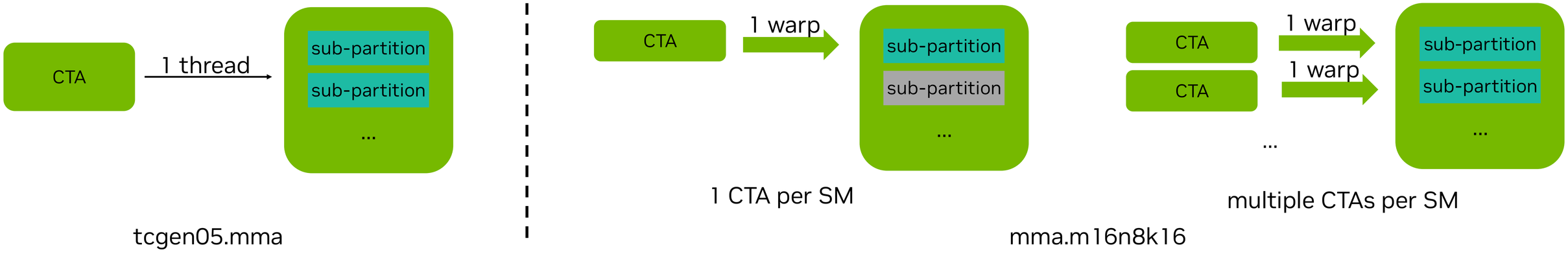
- Softmax warp-group: 4 warps, 2 stages
- Ping-pong scheduling: iterate over S0/P0 and S1/P1 sequentially and repetitively
- Softmax operation can be overlapped by two gemm



Overlapped by Gemm

Using Warp-Level MMA Instruction for Small GQA

- Problem:
  - Small GQA (<= 16) produces very narrow Q tiles, leading to poor utilization with SM-wide MMA(tcgen05.mma)
- Instruction behavior:
  - tcgen05.mma: issued by a single thread, executed by the entire SM
  - mma.m16n8k16: issued by a single warp, executed by one SM execution slice(sub-partition)
- Key Idea:
  - Switch to warp-level MMA for finer-grained execution
- Result:
  - Multiple warps from multiple CTAs(Thead Blocks) can issue MMA concurrently on the same SM, significantly increasing parallelism



Forward Performance

- NVIDIA B200
- Frequency: 3996Mhz, 1845Mhz
- CUTLASS Python implementation vs Triton based implementation

mma.m16n8k16
For small GQA size

seq_len	gqa	h_k	topk	batch_szie	Triton(ms)	CUTLASS Python(ms)	speedup vs triton
32768	8	1	16	4	7.41	3.77	1.96
32768	8	1	16	8	14.88	7.44	2.00
32768	8	1	16	16	36.44	14.81	2.46
32768	16	1	16	4	7.41	4.26	1.74
32768	16	1	16	8	14.89	8.36	1.78
32768	16	1	16	16	36.56	16.61	2.20

tcgen05.mma
For large GQA size

seq_len	gqa	h_k	topk	batch_szie	Triton(ms)	CUTLASS Python(ms)	speedup vs triton
8192	32	1	16	4	2.93	2.15	1.36
8192	64	1	16	4	2.91	2.16	1.35
8192	128	1	16	4	6.01	2.21	2.71
16384	32	1	16	4	5.84	4.27	1.37
16384	64	1	16	4	5.81	4.31	1.35
16384	128	1	16	4	12.00	4.39	2.73
32768	32	1	16	4	11.66	8.56	1.36
32768	64	1	16	4	11.65	8.61	1.35
32768	128	1	16	4	24.06	8.79	2.74

Selection Attention Backward

Given kv_block_size = 64 and gqa_size = 16, d=128

```
Given kv_block_size = 64 and gqa_size = 16, d=128
for j in range(Tc):
    for i in range(Tr):
        St_j_i = K_j @ Q_i.T * softmax_scale  --> (64, 128) @ (128, 16)
        Pt_j_i = torch.exp(St_j_i - lse_i.T)
        dV_j += Pt_j_i @ dO_i                 --> (64, 16) @ (16, 128)
        dPt_j_i = V_j @ dO_i.T                --> (64, 128) @ (128, 16)
        dSt_j_i = Pt_j_i * (dPt_j_i - D_i.T) * softmax_scale
        dQ_i = dS_i_j @ K_j                   --> (16, 64) @ (64, 128)
        dK_j += dSt_j_i @ Q_i                 --> (64, 16) @ (16, 128)
        # write back dQ
        dQ += dQ_i
    # write back dK, dV
    dK += dK_j
    dV += dV_j
```

Red: cannot pack
Green: can be packed

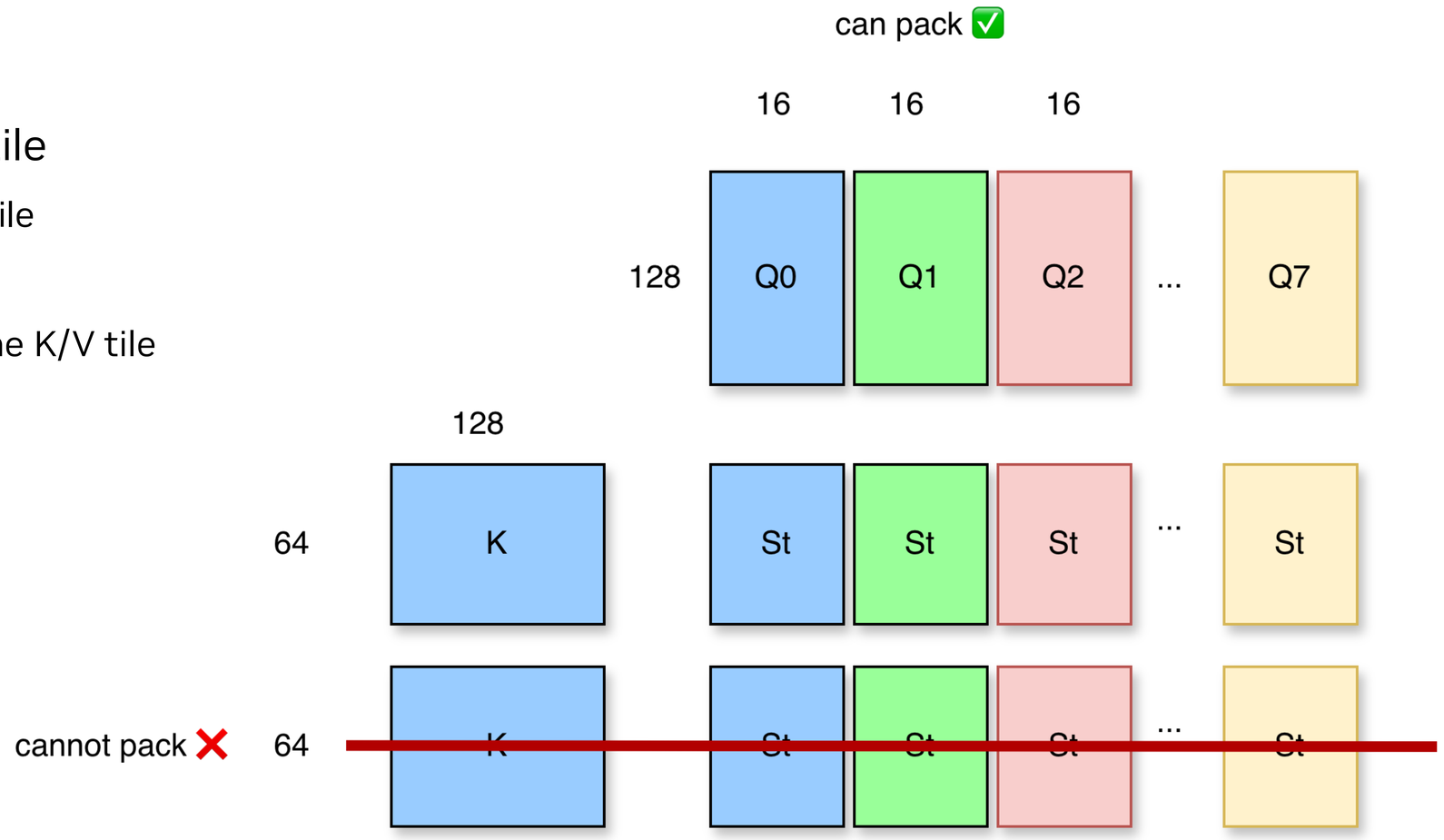
Design challenges in the backward kernel

- Outer loop -> KV loop
  - This loop is in parallel (across different CTAs), and each CTA loads only 1 K tile and 1 V tile.
  - For standard MHA, different Q tile attends to same KV tile(with causal mask), vice versa
  - In selection attention, each Q tile([gqa_size, d]) attend to different K/V tile, so we cannot pack K/V tile for better tensor core utilization
- Inner loop -> Q loop
  - This loop is iterating inside each CTA
  - For standard MHA, Q tile can be size of [128, 128].
  - In selection attention, each Q tile is size of [gqa_size, 128], we can pack multiple Q tiles into one

Multiple Tile Packing

Q tile attention to different K/V tile

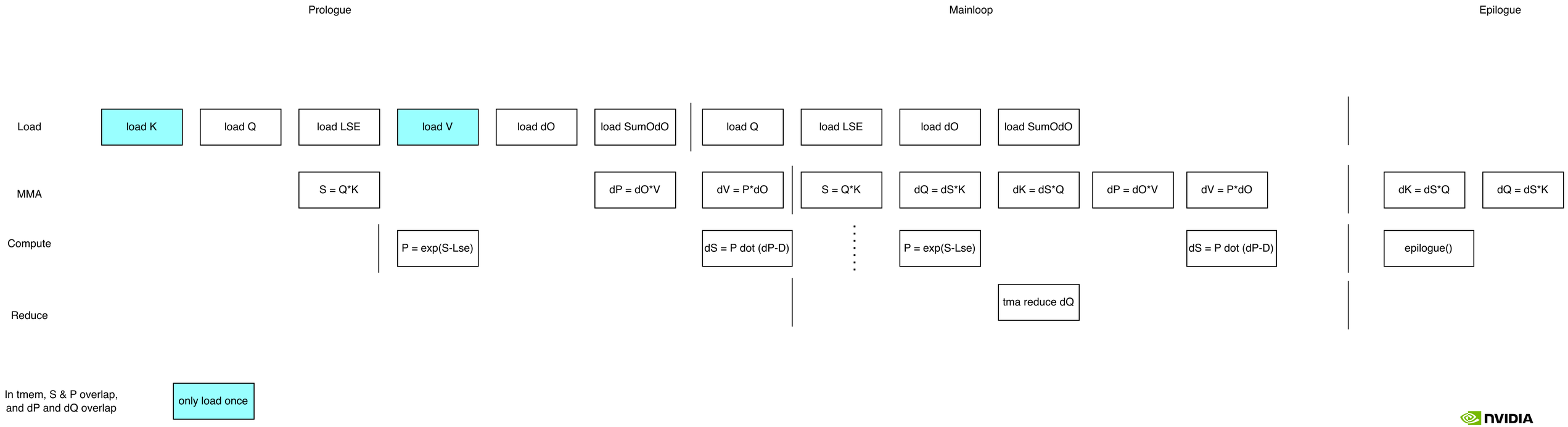
- --> K/V tile attention to different Q tile
- Cannot pack K/V tile
- Pack Q tile which attends to the same K/V tile



Timeline of Backward Kernel

Warp specialized:

- Load: 1 warp
  - Load K,Q,V, Lse, dO, SumOdO
- Mma: 1 warp
  - 5 gemm
- Compute: 4 warps
  - Compute P and dS
- Reduce: 4 warps
  - Copy dQ, tmem->register->smem->gmem



Backward Performance

- NVIDIA B200
- Frequency: 3996Mhz, 1845Mhz
- CUTLASS Python implantation vs Triton based implementation

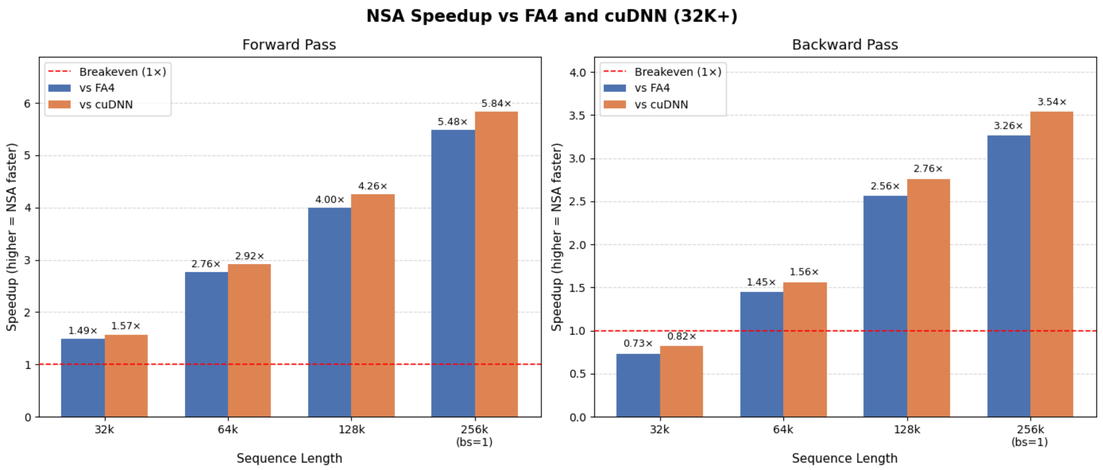
seq_len	gqa	h_k	topk	batch_size	Triton(ms)	CUTLASS Python(ms)	speedup vs triton
1024	16	1	16	16	18.09	10.35	1.75
1024	16	2	16	16	41.18	11.11	3.71
2048	16	1	16	16	35.98	12.62	2.85
2048	16	2	16	16	82.29	15.37	5.35
4096	16	1	16	16	72.63	16.48	4.41
4096	16	2	16	16	167.30	23.42	7.14
8192	16	1	16	16	149.32	23.87	6.26
8192	16	2	16	16	340.48	38.80	8.78
16384	16	1	16	16	301.37	39.24	7.68
16384	16	2	16	16	686.21	69.60	9.86
32768	16	1	16	16	605.03	69.67	8.68
32768	16	2	16	16	1,376.58	131.26	10.49

End-to-End Performance Comparison

End-to-End Performance Comparison

Parameter	Value
H_q	64
H_k	4
Head_dim	128
TopK	16
Selection Block Size	64
Compression Block Stride	32
Compression Block Size	32
SWA Window Size	512
MMA Tiler Compression	(128, 64)
MMA Tiler TopK, Selection, SWA	(128, 128)
Batch size	2 (1 at 256K)
Dtype	BF16

Seq Len	Fwd speedup vs FA4	Fwd speedup vs cuDNN	Bwd speedup vs FA4	Bwd speedup vs cuDNN
32K	1.49 x	1.57 x	0.73 x	0.82 x
64K	2.76 x	2.92 x	1.45 x	1.56 x
128K	4.00 x	4.25 x	2.56 x	2.76 x
256K (bs=1)	5.48 x	5.835 x	3.26 x	3.54 x



NVIDIA